COMPUTER ORGANIZATION

Subject Code : _____

UNIT – III

Computer Arithmetic & Processor Organization

Syllabus

Computer Arithmetic: Addition and Subtraction of Signed Numbers, Design of Fast Adders, Multiplication of Positive Numbers, Signed-operand Multiplication, Fast Multiplication, Integer Division, Floating-Point Numbers and Operations.

Processor Organization: Fundamental Concepts, Execution of a Complete Instruction, Multiple-Bus Organization, Hardwired Control and Microprogrammed Control.

Index

Topic	Page
1. Addition & Subtraction of Signed Numbers	2 – 3
2. Design of Fast Adders	4 – 5
3. Multiplication of Positive Numbers	6 – 7
4. Signed-operand Multiplication (Booth)	8 – 9
5. Fast Multiplication	10 – 11
6. Integer Division	12 – 13
7. Floating-Point Numbers & Operations	14 – 15
8. Processor Fundamentals & Instruction Execution	16 – 17
9. Multiple-Bus Organization	18
10. Hardwired Control	19
11. Microprogrammed Control	20

1. Addition & Subtraction of Signed Numbers

Representation of signed numbers (n bits): the MSB is the sign bit (0 = positive, 1 = negative). Three methods:

Method	Negative of +5 (4-bit)	Range	Zero
Sign-magnitude	1101	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	two (±0)
1's complement	1010	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	two (±0)
2's complement	1011	$-2^{n-1} \dots +(2^{n-1}-1)$	one (0)

2's complement (most used)

- Reason: one adder works for both positive and negative numbers; only one zero.
- To negate: invert all bits (1's complement) and add 1. Ex: +5 = 0101 → 1010 → +1 = 1011 = −5.
- Value = $-b_{n-1}\cdot 2^{n-1} + \Sigma\, b_i \cdot 2^i$ $(i = 0 \dots n-2)$.
- Sign extension: copy the sign bit to the left to widen a number (1011 → 11111011).

Addition rules (2's complement)

- Add the two n-bit numbers bit by bit, carries moving left.
- Discard the carry out of the MSB. No correction step is needed.
- Result is correct if no overflow occurs.

Ex 1: (+3) + (−5) = 0011 + 1011 = 1110 = −2 ✔

Ex 2: (−3) + (−2) = 1101 + 1110 = 1 1011 → drop carry → 1011 = −5 ✔

Ex 3: (+6) + (+2) = 0110 + 0010 = 1000 → ?? overflow!

Subtraction & Overflow

Subtraction

$X - Y = X + (2\text{'s complement of } Y) = X + \bar{Y} + 1$. So the same adder is used: invert Y with XOR gates and apply 1 to the carry-in c_0.

Ex: (+5) − (+3): 0101 + (1100 + 1) = 0101 + 1101 = 1 0010

Discard carry → 0010 = +2 ✔

Overflow

Result lies outside the range of n bits $-2^{n-1} \ldots 2^{n-1}-1$.

- Occurs only when adding numbers of the SAME sign and the result has the OPPOSITE sign.
- (+ve) + (−ve) can never overflow.
- Detection: Overflow = $c_n \oplus c_{n-1}$ (carry into MSB XOR carry out of MSB).
- Or by sign bits: $V = x_{n-1}\, y_{n-1}\, \bar{s}_{n-1} + \bar{x}_{n-1}\, \bar{y}_{n-1}\, s_{n-1}$.

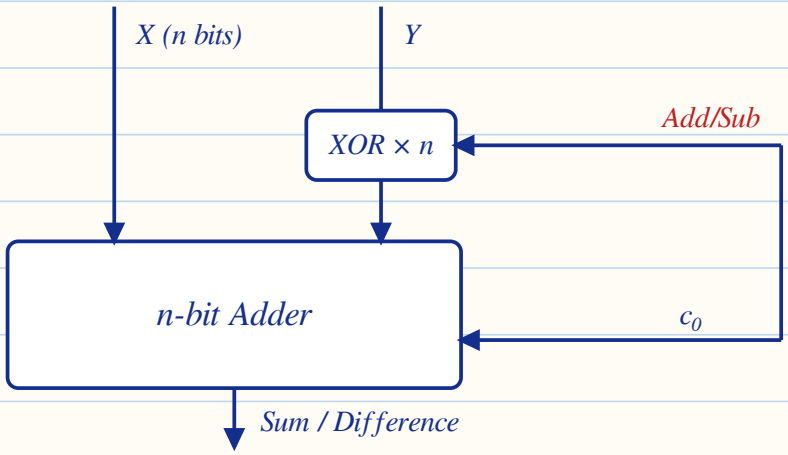
Ex: 0111 (+7) + 0101 (+5) = 1100 → carry into MSB = 1, carry out = 0 → Overflow = 1 ✔ (true sum 12 > 7).

Add / Subtract unit (logic diagram)

Add/Sub = 0 → X + Y

Add/Sub = 1 → X − Y

(Y inverted, c_0=1)



Add/Sub control line selects operation; the XOR gates pass Y unchanged (0) or complement it (1). Overflow flag generated from $c_n \oplus c_{n-1}$.

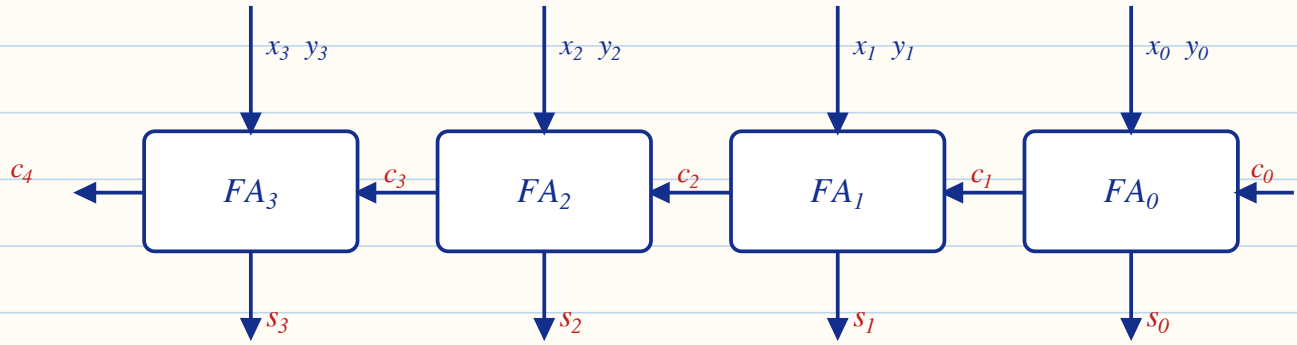
2. Design of Fast Adders

Full Adder (FA)

$s_i = x_i \oplus y_i \oplus c_i \qquad c_{i+1} = x_i\, y_i + x_i\, c_i + y_i\, c_i$

n-bit Ripple-Carry Adder

n full adders in series; carry c_{i+1} of one stage is the carry-in of the next.



- *Delay: each FA needs 2 gate delays for carry → total ≈ 2n gate delays (last sum ≈ 2n+1).*
- *Problem: slow for large n (32/64-bit) because every carry waits for the previous stage.*
- *Solution: compute carries in advance → Carry-Lookahead Adder.*

Carry-Lookahead Adder (CLA) – derivation

Rewrite the carry: $c_{i+1} = x_i y_i + (x_i + y_i)\, c_i \;=\; G_i + P_i\, c_i$

- *Generate* $G_i = x_i\, y_i$ *(stage creates a carry on its own).*
- *Propagate* $P_i = x_i + y_i$ *(stage passes an incoming carry).*

Expanding the recurrence repeatedly:

$$c_1 = G_0 + P_0 c_0$$

$$c_2 = G_1 + P_1 G_0 + P_1 P_0 c_0$$

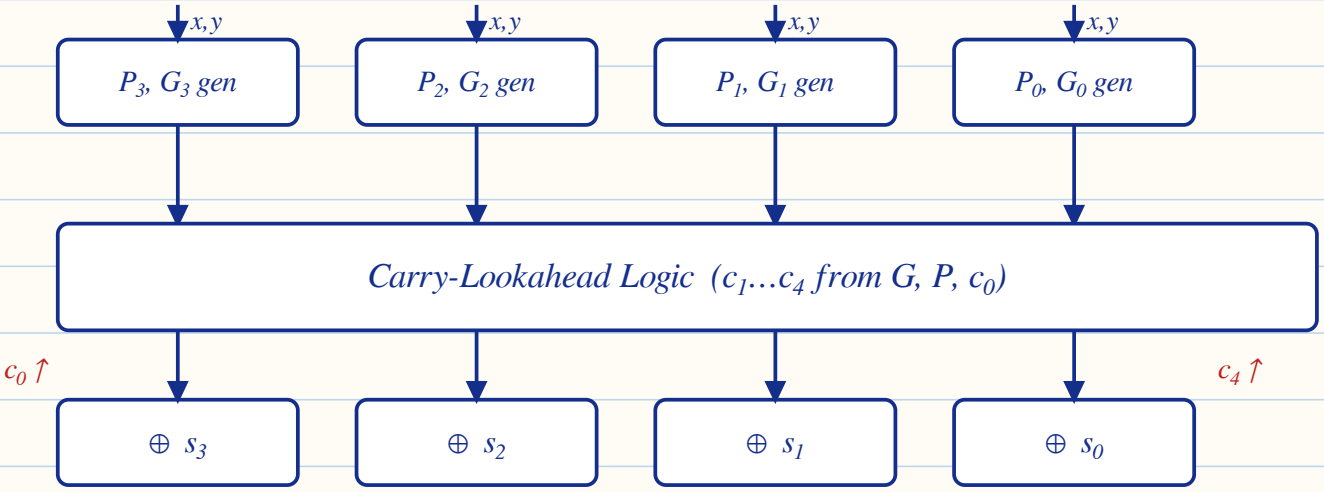
$$c_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0$$

$$c_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0$$

Carry-Lookahead Adder (contd.)

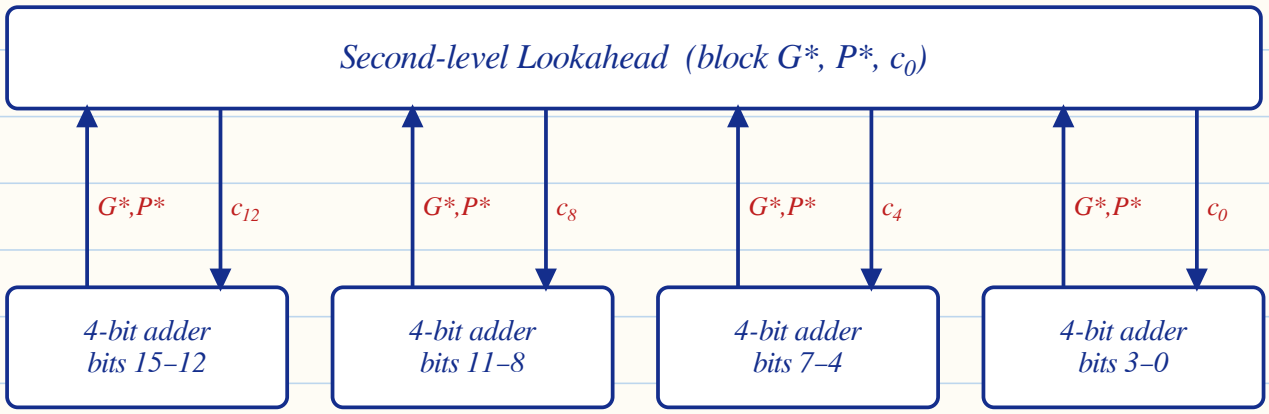
All carries depend only on x, y and c_0, so they are produced in PARALLEL by one AND-OR network. Sum: $s_i = P_i' \oplus c_i$, with $P_i' = x_i \oplus y_i$.

4-bit CLA block



- *Delay: PG generation 1 gate, carry logic 2 gates, sum XOR 1 gate → about 4 gate delays, independent of n for the block.*

Cascading blocks (16-bit / 32-bit adder)



- *Block generate $G^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$*
- *Block propagate $P^* = P_3P_2P_1P_0$*

Adder	Delay (n-bit)	Hardware
Ripple-carry	≈ 2n gate delays	Least
CLA (blocked)	≈ 8 for 32-bit	More

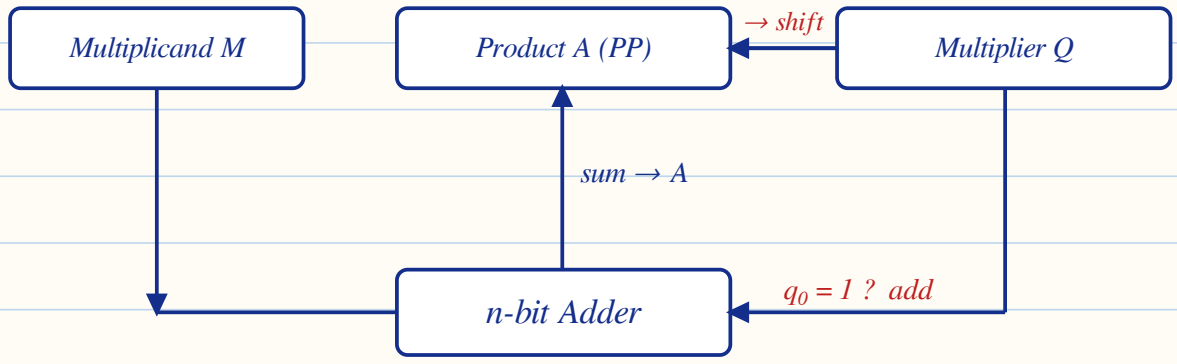
3. Multiplication of Positive Numbers

Multiplying an n-bit multiplicand M by an n-bit multiplier Q gives a 2n-bit product.

Paper-and-pencil method: for each multiplier bit q_i write M (if q_i=1) or 0 (if q_i=0), shifted left i places, and add all partial products.

```
    1101   (13)  Multiplicand M
×   1011   (11)  Multiplier Q
 ------
    1101
   1101
  0000
 1101
--------
10001111   (143)
```

Sequential (shift-and-add) multiplier



- *Registers: M (multiplicand), Q (multiplier), A (partial product, init 0), carry flip-flop C, counter n.*
- *Repeat n times: if $q_0 = 1$ then $A = A + M$; then shift C,A,Q right one bit.*
- *At the end the 2n-bit product is in A:Q. Time ≈ n additions.*

Worked Example & Array Multiplier

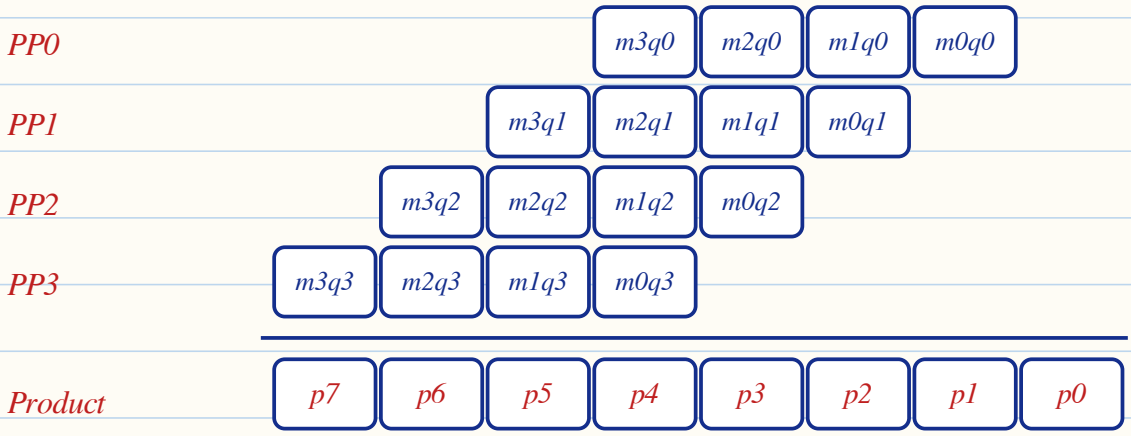
13 × 11, with M = 1101, Q = 1011, n = 4:

Step	Action	C	A	Q
Init	—	0	0000	1011
1	q0=1: A=A+M, shift	0	0110	1101
2	q0=1: A=A+M, shift	0	1001	1110
3	q0=0: shift only	0	0100	1111
4	q0=1: A=A+M, shift	0	1000	1111

Product = A:Q = 1000 1111 = 143 ✔

Array (combinational) multiplier

AND gates give partial products $PP_i = M \cdot q_i$;
FA array adds column-wise (diagonal ripple).



- *Each cell = AND gate + full adder; n×n array of cells (n^2 cells).*
- *Delay ≈ (2n − 1) FA delays (longest path along diagonal and bottom row) → very regular layout, good for VLSI.*
- *Faster than sequential (no clock steps) but needs much more hardware.*

Important Questions

Q *Explain the sequential multiplier with 13 × 11.*

Q *Write a note on the array multiplier.*

4. Signed-operand Multiplication

Shift-and-add fails for negative 2's-complement numbers (sign bit has negative weight). Simple fix: multiply magnitudes and fix the sign — slow. Better: Booth's algorithm.

Booth's Algorithm – idea

A run of 1s in the multiplier, …0 1 1 1 1 0… $= 2^{k+1} - 2^{j}$. So instead of adding M for every 1 bit, do ONE subtraction at the start of the run and ONE addition just after its end.

0 0 1 1 1 1 0 = $+2^5 - 2^1$ → (+32 − 2 = 30)

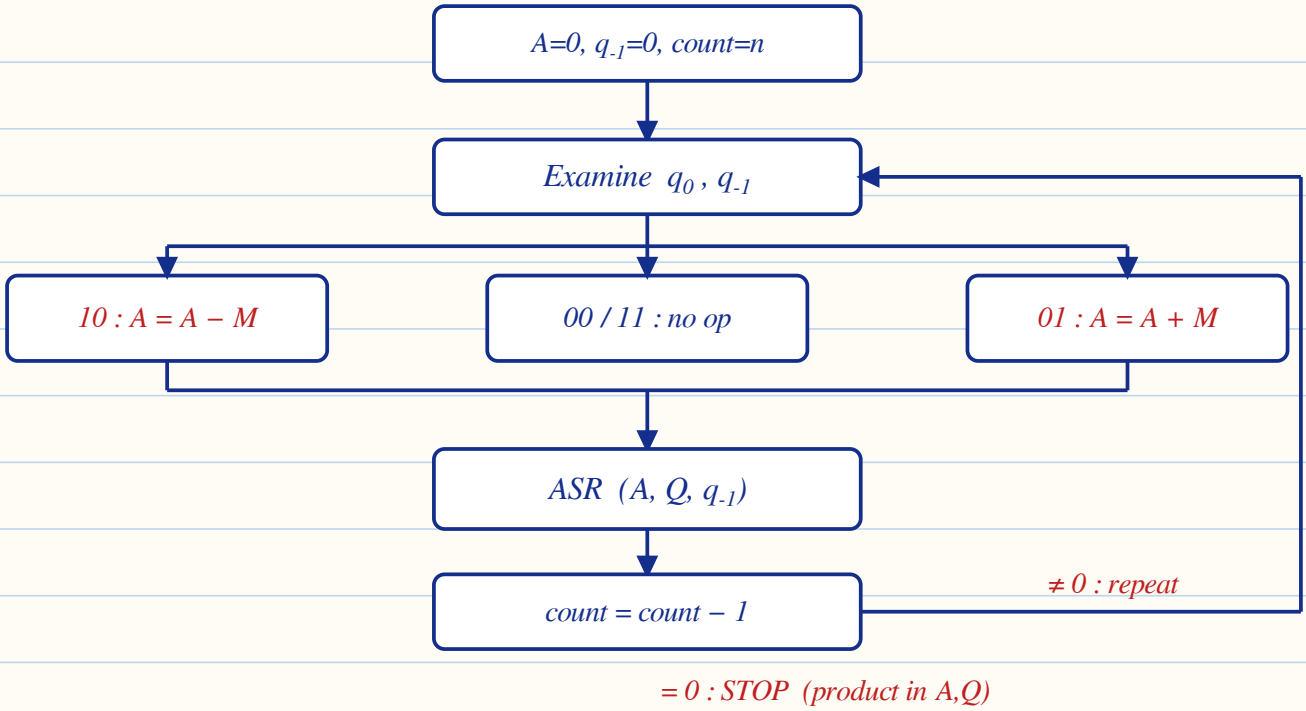
Fewer add/sub operations when the multiplier has long runs of 1s.

Recoding table (scan q_0 and q_{-1}, q_{-1} = 0 initially)

q0	q–1	Action on A	Booth bit
0	0	none, shift	0
0	1	A = A + M	+1
1	0	A = A − M	−1
1	1	none, shift	0

- *After each step perform an ARITHMETIC right shift of A, Q, q_{-1} (sign bit of A is preserved).*
- *Works for +ve and −ve multiplicand and multiplier — no correction required.*
- *Worst case: alternating bits 0101… → n operations; best: all 0s / 1s.*
- *Advantage: handles signs automatically; avg. speed gain for runs of 1s.*

Booth's Algorithm – Flowchart & Example



Ex: (+7) × (–3), n = 4: M = 0111, –M = 1001, Q = 1101

Step	q0 q–1	Operation	A	Q	q–1
Init			0000	1101	0
1	1 0	A=A−M, ASR	1100	1110	1
2	0 1	A=A+M, ASR	0001	1111	0
3	1 0	A=A−M, ASR	1101	0111	1
4	1 1	ASR only	1110	1011	1

Result A:Q = 1110 1011 = –21 ✔ (7 × –3 = –21)

Important Questions

Q *Explain Booth's algorithm with a flowchart; multiply +7 × −3.*

Q *What are the advantages of Booth's algorithm?*

5. Fast Multiplication

Two techniques speed up multiplication: (a) reduce the NUMBER of partial products (bit-pair recoding), (b) add partial products FASTER (carry-save addition).

(a) Bit-pair recoding of the multiplier (Booth radix-4)

Group the Booth-recoded multiplier into pairs → number of summands is halved (n/2). Each pair (q_{i+1}, q_i) with q_{i-1} selects one multiple of M:

q(i+1) q(i) q(i–1)	Multiple of M	Booth pair
0 0 0	0	0
0 0 1	+ M	+1
0 1 0	+ M	+1
0 1 1	+ 2M	+2
1 0 0	– 2M	–2
1 0 1	– M	–1
1 1 0	– M	–1
1 1 1	0	0

- *Shifts by 2 bits per step; 2M is just M shifted left one place (no extra hardware).*

Ex: multiplier Q = 1010 (–6), q_{-1} = 0. Pair 1 = (q1,q0,q-1) = 1 0 0 → –2M. Pair 2 = (q3,q2,q1) = 1 0 1 → –M (weight 4). Total = –2M – 4M = –6M ✔

Carry-Save Addition (CSA) & Wallace Tree

Adding k partial products with ripple adders takes (k−1) full carry propagations. CSA avoids propagating carries: each FA column takes 3 input bits and gives a Sum bit and a Carry bit (kept SEPARATE, carry shifted one place left).

X = 1 0 1 1

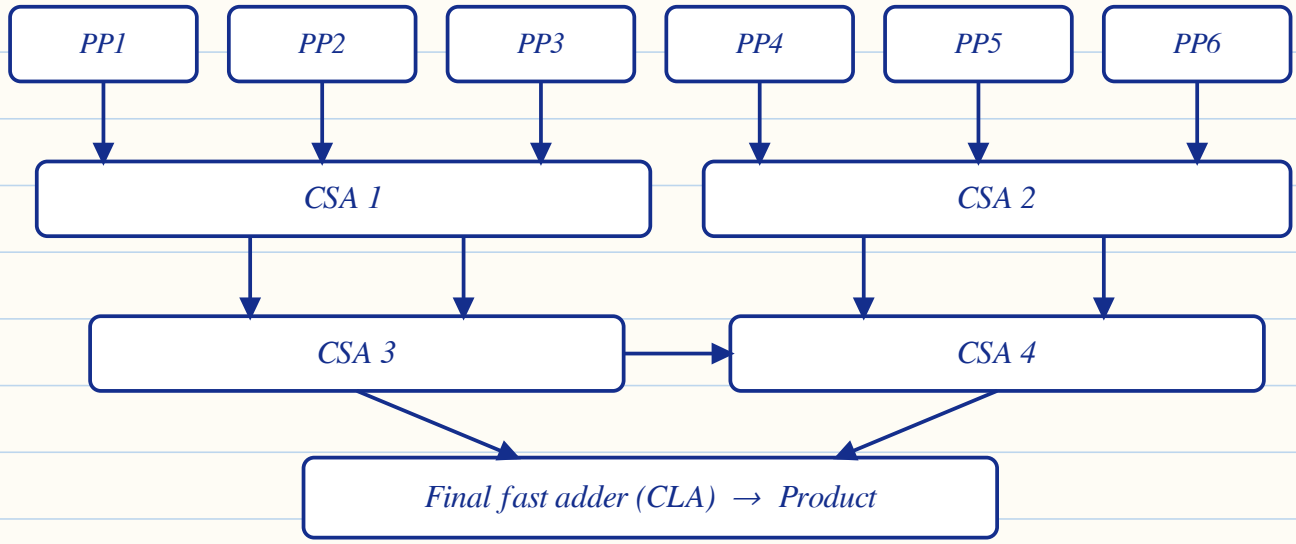
Y = 0 1 1 0

Z = 1 1 0 1

Sum = X⊕Y⊕Z (save)

Carry = majority, shifted left

Wallace / CSA tree

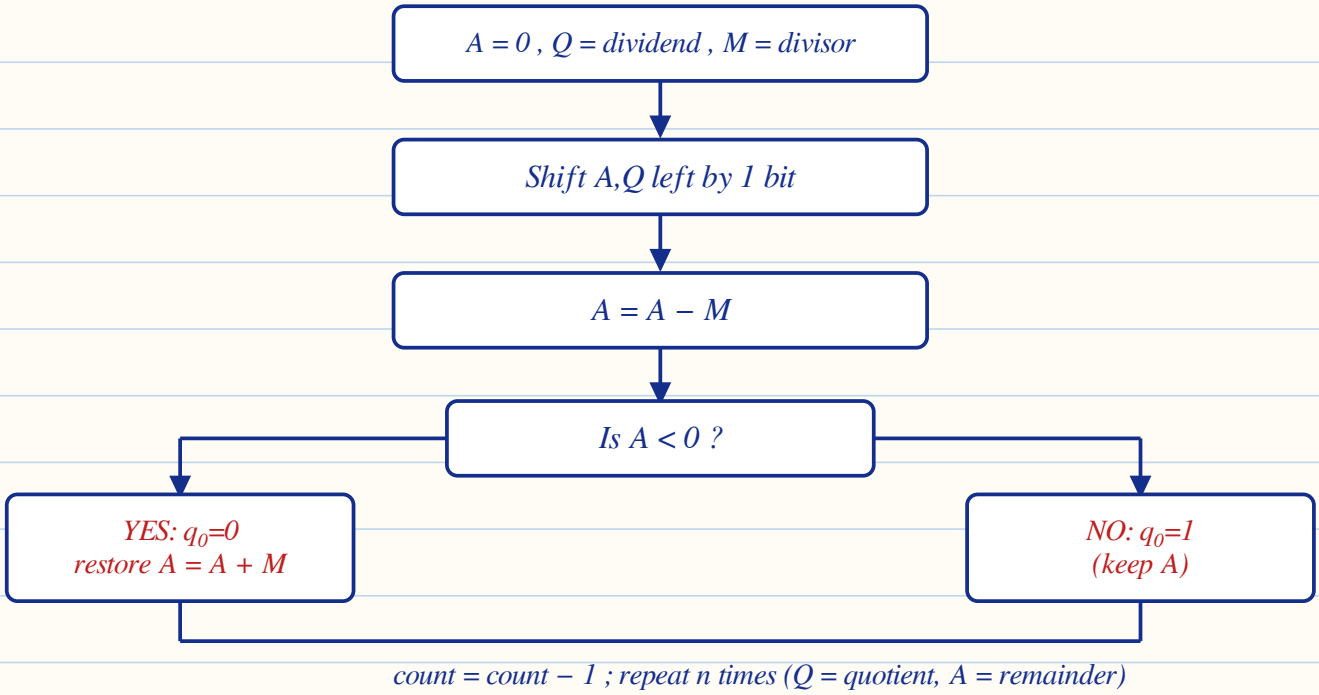


- *3 rows → 2 rows at every CSA level; with k PPs the number of levels ≈ $\log_{1.5}(k/2)$.*
- *Only the FINAL two rows are added with a fast (CLA) adder → delay ∝ log n instead of n.*
- *Combine with bit-pair recoding: n/2 partial products, then CSA tree → fastest practical multiplier.*
- *Disadvantage: irregular wiring, more area than array multiplier.*

6. Integer Division

Dividend = Quotient × Divisor + Remainder. Registers: A (n+1 bits, remainder, init 0), Q (dividend, then quotient), M (divisor), count n.

Restoring division



- *Each cycle: shift left A:Q, subtract M from A; if result is negative put back M (restore) and q_0=0, else q_0=1.*
- *Needs an extra add (restore) → slow when many restores.*

Non-restoring division

Avoids the restore step. Rule per cycle:

- *If A ≥ 0: shift left A:Q, then A = A − M.*
- *If A < 0: shift left A:Q, then A = A + M.*
- *After the operation: A ≥ 0 → q_0=1, else q_0=0.*
- *At the end, if A < 0 add M once to correct the remainder.*

Division – Worked Examples

Restoring: 8 ÷ 3 (n = 4, M = 0011)

Step	A:Q after shift	A−M	Result	A	Q
Init				0000	1000
1	0001 000_	1110 (<0)	q0=0, restore	0001	0000
2	0010 000_	1111 (<0)	q0=0, restore	0010	0000
3	0100 000_	0001 (≥0)	q0=1	0001	0001
4	0010 001_	1111 (<0)	q0=0, restore	0010	0010

Quotient = 0010 = 2, Remainder = 0010 = 2 ✔ (8 = 3 × 2 + 2)

Comparison

Point	Restoring	Non-restoring
Operations / cycle	sub + (maybe) add	one add OR sub
Speed	slower	faster
Final correction	none	add M if A<0
Hardware	simple	slightly more control

- Overflow / divide-by-zero must be detected before division starts (divisor = 0 → error flag).
- Signed division: divide magnitudes, quotient sign = sign(dividend) ⊕ sign(divisor), remainder takes the dividend's sign.

Important Questions

Q Explain restoring and non-restoring division with an example.

Q Differentiate restoring vs non-restoring division.

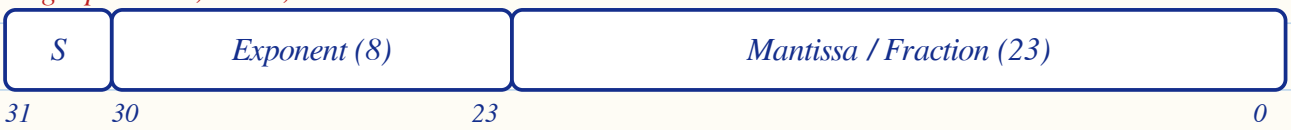
7. Floating-Point Numbers & Operations

Scientific form to represent very large and very small numbers: $\pm 1.M \times 2^{E-bias}$.

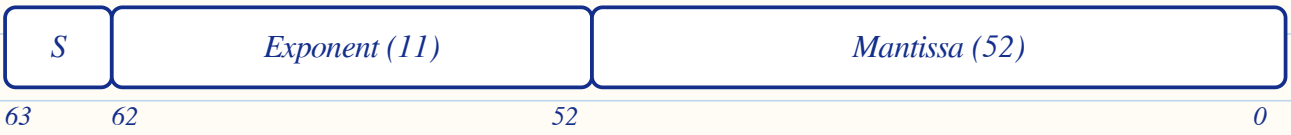
A floating-point number stores Sign, Exponent, Mantissa.

IEEE 754 formats

Single precision, 32 bit, bias = 127



Double precision, 64 bit, bias = 1023



- *Normalized: leading 1 is implied (hidden bit) → 1.M.*
- *Exponent stored in excess (biased) form: E = true exponent + 127 (single) or + 1023 (double).*
- *Value* $= (-1)^S \times 1.M \times 2^{E-127}$.

Example: represent 12.375

$12.375 = 1100.011_2 = 1.100011 \times 2^3 \rightarrow S = 0, E = 3+127 = 130 = 10000010, M = 10001100\ldots0$

IEEE word: 0 | 10000010 | 10001100000000000000000 = 0x41460000

Special values (single precision)

Exponent	Mantissa	Meaning
0000 0000	0	± 0
0000 0000	≠ 0	denormalized
1111 1111	0	± ∞
1111 1111	≠ 0	NaN (not a number)

Floating-Point Operations

Add / Subtract

Floating-point add/subtract : 5 steps



- *1) Compare exponents, difference $d = |E_1 - E_2|$. 2) Shift mantissa of the smaller number right d places.*
- *3) Add / subtract mantissas (sign handling). 4) Normalize result (shift, adjust exponent). 5) Round.*

Ex: $1.5 + 0.25 = 1.1\times2^0 + 1.0\times2^{-2}$. Align: 0.01×2^0. Add: $1.1 + 0.01 = 1.11\times2^0 = 1.75$

✔

Multiply / Divide

- *Multiply: $E = E_1 + E_2 - bias$; $M = M_1 \times M_2$; $S = S_1 \oplus S_2$; normalize, round.*
- *Divide: $E = E_1 - E_2 + bias$; $M = M_1 \div M_2$; $S = S_1 \oplus S_2$; normalize, round.*

Guard bits & rounding

Extra bits kept during alignment: Guard (g), Round (r), Sticky (s = OR of all bits shifted beyond r). Rounding modes:

- *Chop (truncate) • Round to nearest even (default) • Toward $+\infty$ • Toward $-\infty$*

Exceptions

- *Exponent overflow (result too big → ∞), underflow (too small → 0/denormal), inexact, invalid (0/0, $\infty-\infty$ → NaN), divide-by-zero.*

Important Questions

Q *Explain IEEE 754 format; represent 12.375.*

Q *Explain floating-point addition steps.*

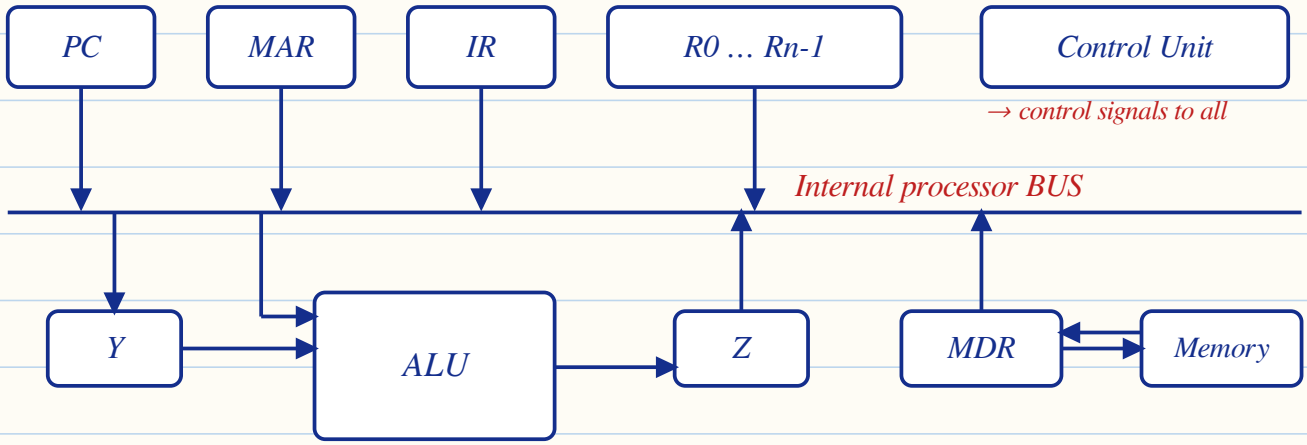
8. Processor Organization – Fundamental Concepts

Processor = Datapath (registers + ALU + buses) + Control unit. It executes instructions in fetch–decode–execute cycles.

Main registers

- *PC – address of next instruction; IR – holds current instruction; MAR – memory address; MDR – memory data;*
- *R0…Rn-1 – general registers; Y, Z – temporary registers around the ALU.*

Single-bus datapath



Only ONE register can put data on the bus per clock step (others only read).

Register transfers use gating: R_{out} puts a register on the bus, R_{in} loads it from the bus (control signals). Example: PC_{out}, MAR_{in}.

- *Clock period ≥ time for one bus transfer + ALU + setup.*
- *Memory read: MAR_{in}, Read, wait for MFC (Memory Function Completed), then MDR_{inE} → IR.*

Important Questions

Q *Explain the single-bus datapath of a processor.*

Q *Write the control sequence for fetching an instruction.*

Execution of a Complete Instruction

Instruction cycle = Fetch + Decode + Execute + Store. Example: ADD R1, R2 (single bus), and a branch.

Fetch (common to all)

Step	Control signals	Action
1	PC_out, MAR_in, Read, Select4, Add, Z_in	MAR ← PC; Z ← PC+4
2	Z_out, PC_in, Y_in, WMFC	PC ← Z; wait for memory
3	MDR_out, IR_in	IR ← instruction
4	—	decode opcode

Execute: ADD R1, R2 (R1 ← R1 + R2)

Step	Control signals	Action
4	R1_out, Y_in	Y ← R1
5	R2_out, Select Y, Add, Z_in	Z ← Y + R2
6	Z_out, R1_in, End	R1 ← Z

Execute: unconditional BRANCH offset

Step	Control signals	Action
4	Offset_field_of_IR_out, Add, Z_in	Z ← PC + offset
5	Z_out, PC_in, End	PC ← target

- Conditional branch: step 4 tests condition codes; if false, the End signal is generated early.
- Load/Store need extra steps for memory access of the operand through MAR/MDR.

Important Questions

Q Write the control steps of ADD R1,R2 and a branch.

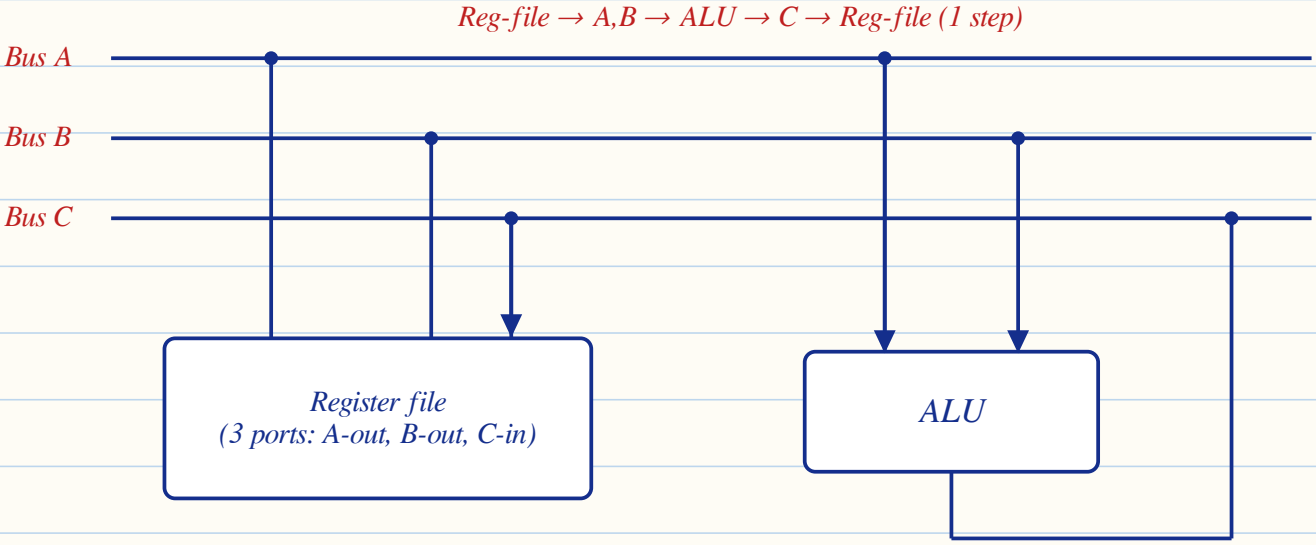
Q Explain how the multiple-bus organization increases speed.

9. Multiple-Bus Organization

In a single-bus processor only one data transfer per clock → many steps per instruction.

Providing several buses allows parallel transfers.

Three-bus organization



- *Bus A and Bus B carry the two ALU operands out of a register file with two read ports; Bus C carries the result back to the register file (one write port).*
- *Registers Y and Z are NOT needed; a separate incrementer updates PC (PC ← PC + 4) in parallel.*
- *Add R4, R5, R6 : R4_out→A, R5_out→B, ALU Add, R6_in← C → just ONE step.*

Comparison

Feature	Single bus	Three bus
Transfers / cycle	1	up to 3
Steps for ADD	≈ 3	1
Hardware cost	low	high
Performance	lower	higher

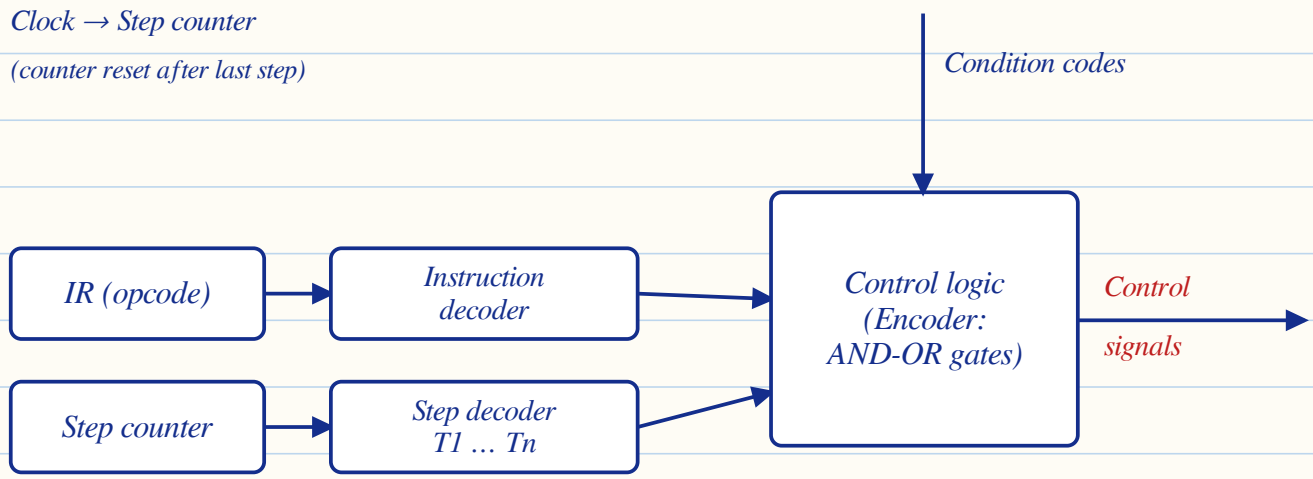
Important Questions

Q *Explain the three-bus organization with a diagram.*

Q *Compare single-bus and multiple-bus processors.*

10. Hardwired Control

Control signals are produced by a fixed sequential logic circuit (gates, decoders). Fastest control; used in RISC processors.



Inputs and outputs

- *Inputs: control step counter (decoded to $T_1 \dots T_n$), instruction decoder ($INS_1 \dots INS_m$ from IR), condition codes, external inputs (interrupts).*
- *Outputs: control signals such as PC_{out}, MAR_{in}, Add, Z_{in}, End.*

Each signal = OR of the (step AND instruction) conditions where it is needed:

$$Z_{in} = T_1 + T_6 \cdot ADD + T_4 \cdot BR + \dots$$

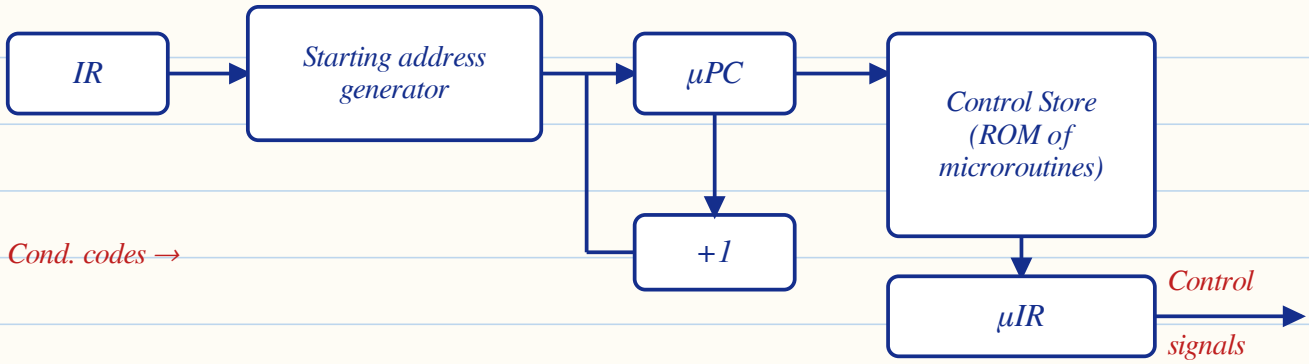
$$End = T_7 \cdot ADD + T_5 \cdot BR + (T_5 \cdot N + T_4 \cdot \bar{N}) \cdot BRN + \dots$$

- *Step counter is reset to T_1 by the End signal.*
- *Advantages: very fast; Disadvantages: hard to design / modify, complex for large instruction sets (CISC).*

11. Microprogrammed Control

Control signals are stored as control words (microinstructions) in a control store (ROM).

The set of microinstructions for one machine instruction = microroutine.



μPC points to next microinstruction; branch address loaded from μIR

Microinstruction formats

Horizontal (1 bit per signal, wide word)

Next addr	PCout	MARin	Read	Zin	Add	…	Yin

Vertical (encoded fields, decoder needed)

Next addr	F1 (4 bit)	F2 (3 bit)	F3 (2 bit)

- *Sequencing: μPC ← starting address from IR opcode; then μPC+1, or branch address (with conditions); fetch microroutine is common.*
- *Wide-branch addressing (bit-ORing) and prefetching of microinstructions improve speed.*

Hardwired	Microprogrammed
Fast, fixed logic	Slower, ROM lookup
Hard to modify	Easy to change/extend
Used in RISC	Used in CISC